

GoalEvolve: From Handcrafted Algorithm Priors to Goal-Driven Evolution of Physical Design Algorithms

Abstract

Physical design algorithms operate within tightly coupled, multi-stage optimization flows, where stage-local gains may vanish or induce downstream degradation. Existing program-evolution frameworks often rely on stage-local objectives or undifferentiated multi-metric feedback, which neither guarantee better final results nor identify which unmet requirement should guide the next iteration. We present GoalEvolve, a goal-driven framework that makes physical design algorithm evolution accountable for the final quality of results (QoR) of the complete flow. Given a multi-objective QoR target region, GoalEvolve converts unmet requirements into normalized target gaps, identifies the dominant bottleneck, and uses stage-resolved checkpoint evidence to locate the responsible stage. An LLM-based Teacher then narrows the search to a relevant algorithmic decision and source region, while parallel Student agents implement and validate hypotheses through full-flow evaluation. Local effects, optimization debt, and downstream retention are retained as mechanism evidence for subsequent evolution. We instantiate GoalEvolve for OpenROAD post-placement optimization. Across eight ASAP7 designs, GoalEvolve reaches all prescribed QoR target regions and yields a 28.83% average improvement in post-route TNS, together with average reductions of 14.31% and 12.23% in leakage and dynamic power, respectively, relative to the default OpenROAD flow. On two representative designs, GoalEvolve reaches the same targets as Codex goal mode with 32.73% fewer valid full-flow evaluations on average.

Keywords

physical design automation, post-placement optimization, timing and power optimization, large language models, program evolution

1 Introduction

Physical design proceeds through a tightly coupled implementation flow consisting of floorplanning, placement, post-placement optimization, routing, and post-routing optimization, in which algorithmic decisions can propagate across stages. A common phenomenon is that an improvement achieved at one stage does not necessarily translate into better final QoR, and may even degrade the design after downstream processing. For example, an aggressive timing transformation after placement may reduce local violations while increasing congestion, routing detours, or power, ultimately worsening post-route QoR. A stage-level decision is therefore valuable only if it preserves design validity and its benefit persists through the complete flow. This property imposes a stringent requirement on physical design algorithm evolution: optimizing an algorithm for a particular stage must account not only for its immediate objective, but also for its impact on the final multi-objective QoR of the entire implementation flow.

The conventional development process follows an algorithm-first paradigm. Researchers typically begin with a perceived weakness, formulate an optimization strategy, implement the corresponding



Figure 1: Comparison of typical EDA evolution and GoalEvolve across three key dimensions. Unlike score-driven selection, GoalEvolve uses the remaining QoR gap to guide source edits and retain mechanism-level evidence.

mechanism, and then evaluate the complete flow. This approach is effective when the relationship between the proposed mechanism and the target QoR is well understood. In many design-closure problems, however, the desired outcome is explicit while the algorithmic path toward it remains unclear. Stage logs and intermediate reports provide useful diagnostic evidence, but translating that evidence into the next source-level hypothesis still requires substantial manual analysis and domain expertise. Algorithm development therefore proceeds through repeated cycles of diagnosis, implementation, and validation, without an automated way to derive the next intervention directly from the remaining target gap.

LLM agents make repository-scale source-code exploration and modification increasingly practical. General program-evolution systems combine program generation with automated evaluation [12, 15, 16, 20], while recent EDA systems extend this paradigm to logic synthesis and multiple stages of physical design [7, 9, 19, 21]. These studies show that LLM agents can iteratively improve EDA algorithms through execution feedback. Nevertheless, their control logic leaves two issues unresolved in tightly coupled physical design flows. First, when candidate selection is based on the objective of the modified stage, an apparent local gain may be lost or overturned by later stages and thus need not improve the full-flow outcome. Second, multiple reported metrics are often presented without an explicit priority. Such feedback can compare candidates, but does not indicate which remaining target deficit should govern the next source revision. As a result, the agent may alternate among

competing objectives or modify mechanisms unrelated to the current bottleneck, leading to an unfocused search and unnecessary full-flow evaluations.

These limitations suggest that physical design algorithm evolution must be accountable for final full-flow QoR rather than for improvements observed only at the stage being modified. Under this full-flow accountability, QoR should serve not merely as a terminal fitness value, but as a control signal that determines what the evolution process should address next. Rather than asking whether a candidate is locally or globally better than its predecessor, the controller should identify the final QoR requirement with the largest unresolved target gap, locate where the gap is introduced or left unrepaired, and focus the next source modification on the corresponding stage and mechanism. This changes multi-objective evolution from an undirected search over aggregate improvements into a target-directed process focused on a specific unresolved final-QoR requirement in each iteration.

Based on this principle, we present GoalEvolve, a goal-driven framework that evolves physical design algorithms toward an explicit multi-objective QoR target region. Given a QoR specification, GoalEvolve converts unmet final-QoR requirements into normalized target gaps: the dominant gap determines *what* to improve, while stage-resolved checkpoint evidence identifies the responsible stage by showing *where* the deficiency is introduced or left unrepaired. Guided by this diagnosis, an LLM-based Teacher narrows the search to the relevant algorithmic decision and bounded source region, while parallel Student agents implement and validate hypotheses. Stage contracts relate each candidate’s expected local effect to temporary optimization debt and specify the downstream evidence needed to verify that the benefit persists through the complete flow. Successful and failed experiments are retained as persistent mechanism evidence to refine promising mechanisms and avoid ineffective interventions. The same controller operates across designs, while each algorithmic path adapts to design-specific targets and observed behavior. GoalEvolve thus provides a reusable methodology for deriving and revising optimization logic from explicit goals and full-flow evidence rather than improving a single optimization algorithm.

The main contributions of this work are summarized as follows:

- We introduce goal-driven physical design algorithm evolution toward explicit multi-objective QoR regions, formalized as target-set reachability over feasible program variants and evaluated by final full-flow QoR rather than stage-local gains.
- We develop a bottleneck-directed loop that identifies the dominant unmet QoR requirement from normalized target gaps, uses stage-resolved checkpoints to locate the responsible stage, and narrows the next intervention to the relevant algorithmic decision and bounded source region.
- We establish evidence-constrained hypotheses and persistent mechanism memory, retaining successful and failed experiments to guide subsequent evolution.
- We instantiate GoalEvolve for OpenROAD post-placement optimization. On eight ASAP7 designs, it reaches all prescribed QoR regions and yields a 28.83% average improvement in post-route TNS, together with average reductions of 14.31% and 12.23% in leakage and dynamic power, respectively, relative to default OpenROAD. On two representative designs, it reaches the same

targets as Codex goal mode with 32.73% fewer valid full-flow evaluations on average.

- We will open-source the GoalEvolve implementation and experimental scripts to support reproducibility and future research.

The remainder of this paper is organized as follows. Section 2 reviews related work on physical design optimization and LLM-based algorithm evolution. Section 3 presents the GoalEvolve framework for evolving physical design algorithms toward explicit QoR targets and its post-placement instantiation. Section 4 reports the experimental validation, and Section 5 concludes the paper.

2 Background and Related Work

This section traces the progression from expert-driven physical design algorithm development to LLM-based program evolution and repository-scale EDA agents. We first review how representative physical design algorithms rely on domain expertise for development and adaptation, and then examine how evolutionary and agentic systems generate, evaluate, and retain source variants.

2.1 Expert-Driven Algorithm Engineering

Recent physical design algorithms have largely advanced through expert-led revision of optimization models and control logic. RePLace introduces constraint-oriented local-density penalties and dynamic step-size adaptation, whereas DREAMPlace maps analytical placement to PyTorch tensor operators and custom GPU kernels [4, 11]. Differentiable timing-driven placement models STA propagation as a differentiable computation to optimize TNS and WNS directly, while FusionSizer jointly optimizes cell locations and gate sizes through continuous relaxation [6, 8]. OpenPhySyn organizes physical synthesis transformations into violation-driven repair procedures, whereas CUGR combines a probabilistic resource model with three-dimensional routing to improve detailed routability [1, 13]. RL-Sizer formulates sequential gate sizing as reinforcement learning driven by timing rewards, while DAGSizer uses directed timing graphs to predict discrete sizing assignments [3, 14]. Both methods require task-specific model training within predefined sizing spaces. These advances improve physical design optimization, but their development and adaptation still require substantial backend design expertise.

2.2 Agentic AI for Algorithm Evolution

The emergence of LLMs has opened a new path for general algorithm evolution by closing the loop among search-space exploration, code generation, and executable evaluation. FunSearch searches over implementations of a user-specified function within a fixed program scaffold, retains valid programs in island populations, and samples higher-scoring variants for subsequent prompts [16]. AlphaEvolve directly edits code and uses an evolutionary database to select parent programs according to feedback from multiple evaluators [15]. EoH co-evolves natural-language ideas and executable heuristics through fitness-based population updates, whereas ReEvo converts pairwise performance differences into immediate and accumulated reflections that guide crossover and elitist mutation [12, 20]. LLaMEA retains the best algorithm in a single-parent loop and feeds runtime fitness and errors back for mutation or redesign [17]. ShinkaEvolve improves sample efficiency through

exploration-aware parent sampling, novelty filtering, and adaptive LLM selection [10]. Together, these systems demonstrate that LLM agents can sustain empirical algorithm evolution through execution feedback and structured retention, providing a practical foundation for extending this paradigm to EDA.

2.3 Agentic EDA and Source Evolution

Building on these advances, Agentic EDA extends program evolution to repository-scale tool development and design-flow optimization. Yao et al. use an LLM genetic flow to evolve three predefined components of global placement, selecting candidates by both placement quality and diversity [19]. AuDoPEDA combines repository graphs with literature-grounded planning to localize OpenROAD edits, then uses hard gates and staged flow evaluation to commit non-regressing patches while returning failures as planning counterexamples [7]. GR-Evolve externalizes router variants and QoR histories as versioned artifacts, then selects among bounded-search candidates using Pareto priorities over downstream routing QoR and runtime [9]. Self-Evolved ABC assigns major synthesis subsystems to specialized agents, while a central planner selects the next subsystem from multidimensional QoR feedback, merges improvements into a champion version, and rolls back regressions [21]. Existing work already uses structured QoR feedback to localize and revise source changes. However, we are not aware of prior work that formalizes the residual to a predefined end-of-flow QoR region as the control signal for assigning stage responsibility and generating the next source-level algorithm hypothesis. GoalEvolve addresses this gap by making the remaining target distance an explicit basis for subsequent evolution.

3 Our Approach

To move beyond algorithm-first development, we introduce GoalEvolve, a framework for goal-driven evolution of physical design algorithms. It couples two feedback loops: an optimization loop that translates the remaining target gap into a bounded source code experiment, and an evidence loop that converts completed experiments into persistent mechanism knowledge. The remaining goal gap therefore acts as an active control signal for deciding what evidence to inspect and which source mechanism to revise, rather than serving only as a terminal score. Figure 2 provides a structural overview, while Algorithm 1 summarizes the corresponding execution loop.

3.1 Goal-Guided Evolution Planning

GoalEvolve evaluates a program variant p on a design d against design-specific QoR targets. After orienting each metric so that lower values are preferred, we define the normalization scale, normalized target violation, and overall goal distance as follows:

$$s_i(d) = \max\{|q_i(p_0, d) - u_i(d)|, \epsilon_i\}, \quad (1)$$

$$r_i(p, d) = \left\lceil \frac{q_i(p, d) - u_i(d)}{s_i(d)} \right\rceil_+, \quad (2)$$

$$D_F(p, d) = \sum_{i=1}^m w_i r_i(p, d), \quad (3)$$

Algorithm 1 Core execution loop of GoalEvolve.

Input: Initial program p_0 , design d , frozen goal G , fixed evaluation flow F , OpenROAD source-mapping *operatorCards*, published mechanism *paperCards*, and evaluation budget B

Output: Best feasible OpenROAD program p_{best}

Variables: $\text{EPD}[p]$ stores the evaluation record and mechanism status of program p ; pool retains feasible programs across rounds; parent is inherited by the next round, whereas p_{best} is the current champion; *diagnosis*, *mechanisms*, and *hypotheses* denote the identified bottleneck, retrieved algorithmic options, and planned branch modifications, respectively; h denotes one hypothesis and *child* its implementation.

```

1:  $\text{EPD} \leftarrow \emptyset$ 
2:  $\text{EPD}[p_0] \leftarrow \text{Evaluate}(p_0, d, F)$ 
3:  $\text{pool} \leftarrow \{p_0\}$ ;  $\text{parent} \leftarrow p_0$ ;  $p_{\text{best}} \leftarrow p_0$ 
4: while Remaining( $B$ ) and not Satisfy( $\text{EPD}[p_{\text{best}}], G$ ) do
5:    $\text{diagnosis} \leftarrow \text{Diagnose}(\text{EPD}[\text{parent}], G, \text{operatorCards})$ 
6:    $\text{mechanisms} \leftarrow \text{Retrieve}(\text{diagnosis}, \text{paperCards}, \text{EPD})$ 
7:    $\text{hypotheses} \leftarrow \text{Plan}(\text{diagnosis}, \text{mechanisms})$ 
8:   for all  $h \in \text{hypotheses}$  in parallel do
9:      $\text{child} \leftarrow \text{Implement}(\text{parent}, h)$ 
10:     $\text{EPD}[\text{child}] \leftarrow \text{Evaluate}(\text{child}, d, F, \text{EPD}[\text{parent}])$ 
11:     $\text{EPD} \leftarrow \text{Update}(\text{EPD}, h, \text{child})$ 
12:     $\text{pool} \leftarrow \text{Retain}(\text{pool}, \text{child}, \text{EPD}[\text{child}])$ 
13:   end for
14:    $p_{\text{best}} \leftarrow \text{Champion}(\text{pool}, \text{EPD})$ 
15:    $\text{parent} \leftarrow \text{Select}(p_{\text{best}}, \text{pool}, \text{diagnosis}, \text{EPD})$ 
16:    $\text{operatorCards} \leftarrow \text{Refresh}(\text{operatorCards}, \text{parent}, \text{EPD}[\text{parent}])$ 
17: end while
18: return  $p_{\text{best}}$ 

```

where $[x]_+ \equiv \max(0, x)$ and m denotes the number of selected QoR metrics. Here, p_0 denotes the baseline program, $q_i(p, d)$ is the QoR value measured by the fixed evaluation flow, and $u_i(d)$ is the corresponding inclusive upper target bound for design d . The positive ϵ_i prevents division by zero when the baseline value equals the target. Consequently, $D_F(p, d) = 0$ if and only if all selected QoR targets are satisfied. A candidate is deemed goal-compliant only if it also passes all required validity checks.

After each round, GoalEvolve identifies the dominant QoR bottleneck from the weighted target gaps and traces it through the white-box checkpoint trajectory. By mapping final metrics to their affecting stages, it compares expected and observed stage effects while accounting for unresolved optimization debt and downstream loss of earlier gains. A compact reflection prompt links the dominant gap to relevant checkpoint and object-level evidence, retrieving supporting artifacts by reference only when needed. This converts the global gap into a focused evidence request for the next source modification.

The Teacher retrieves potential mechanisms from three complementary sources. Source cards map stage behavior to implementation locations and preservation constraints. Paper mechanism cards translate published methods into implementable mechanisms with expected effects and rejection criteria. Historical records from the Evolutionary Program Database in Section 3.3 help avoid ineffective mechanisms and reconsider promising ones.

Guided by the diagnosis and retrieved evidence, the Teacher prioritizes mechanisms most likely to address the responsible stage, balancing new exploration with refinement or integration supported by prior evidence. Each hypothesis links a concrete source modification to its expected stage response and final QoR effect, with a rejection criterion for retention. GoalEvolve thereby narrows an unmet QoR target to a responsible stage, candidate mechanism, and source implementation, which together define the next evolution target. Lines 5–7 of Algorithm 1 summarize this diagnosis–retrieval–planning sequence.

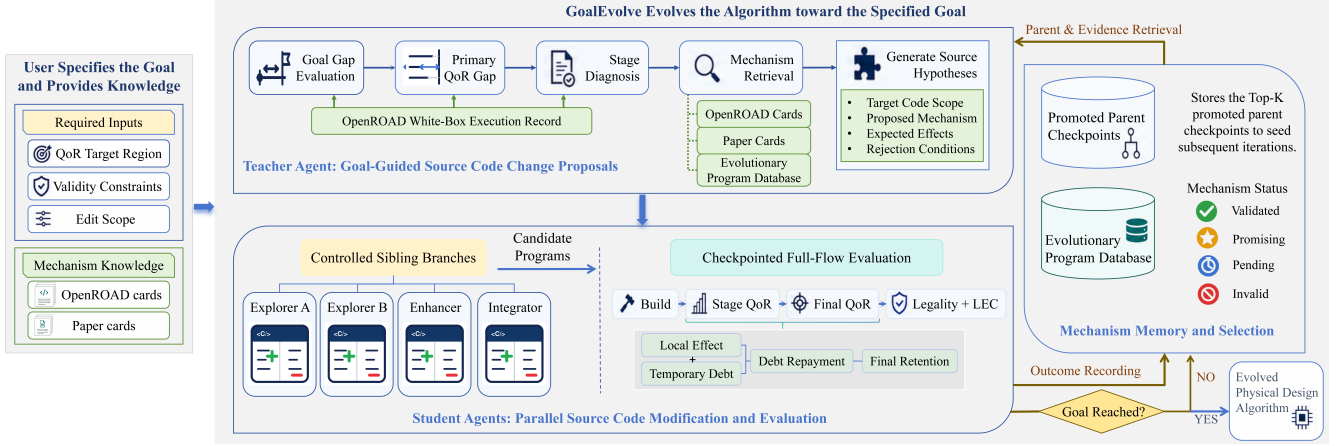


Figure 2: Overview of GoalEvolve. The Teacher converts QoR gaps and retrieved knowledge into targeted source hypotheses, while parallel Students evaluate candidates through checkpointed full-flow runs. Reviewed outcomes update mechanism evidence and parent selection for subsequent iterations.

3.2 Controlled Evolution and Attribution

GoalEvolve evaluates source hypotheses through four controlled Student branches that start from the same parent version. Two branches implement distinct new ideas, one strengthens a mechanism whose gains are promising but not yet stable, and one integrates previously validated mechanisms when their source modifications and expected stage effects are compatible. The common parent and evaluation flow provide a consistent reference for attributing each branch outcome to its assigned hypothesis.

Within each branch, an LLM coding agent with direct repository access and command execution capabilities implements the assigned hypothesis and builds an isolated tool instance. When compilation fails or another implementation issue arises, the agent autonomously diagnoses the problem and revises the patch within a fixed repair budget, preventing an engineering error from prematurely rejecting the underlying mechanism.

GoalEvolve introduces effect-debt analysis, which defines the immediate change in the target QoR metric as the local effect and degradation introduced in other metrics as optimization debt. Both quantities are measured against the common parent after the target stage, while the final comparison measures effect retention and debt repayment. GoalEvolve also tracks object-level netlist changes across checkpoints to identify operations that are later repeated or reversed and therefore require further evolution. These metric and object traces form an evidence signature that binds the source diff to observed behavior for subsequent mechanism selection. Lines 8–13 of Algorithm 1 summarize this controlled evolution-and-attribution process.

3.3 Mechanism Memory and Selection

GoalEvolve builds an Evolutionary Program Database (EPD), which stores all ideas proposed by the Teacher together with an idea tag identifying the optimization target, evaluation records, source patches, and other accumulated feedback. Each record is labeled as *validated*, *promising*, *pending*, or *invalid* according to its evaluation

status and outcome, enabling the EPD to balance exploitation and exploration. For exploitation, compatible validated mechanisms are assigned to the integration Student, while the enhancement Student can perform at most K_{rep} bounded repairs on a promising record. The EPD also retains programs based on distinct mechanisms even when they do not achieve the highest current scores, preventing evolution from being confined to a single path. In parallel, the EPD supports exploration through the implementation and evaluation of newly proposed algorithmic ideas with no matching record in the database. Negative evaluation evidence suppresses repeated proposals and updates the contraindications in mechanism cards. As the database grows, compatibility with the responsible stage and current source state first narrows the candidate set, after which the LLM agent reasons over feedback from previous rounds to select the most promising mechanisms for enhancement or integration.

The evidence accumulated in the EPD also guides checkpoint inheritance. During regular evolution, sibling experiments inherit the champion p^* , defined as the feasible checkpoint with the smallest goal distance, while GoalEvolve retains a top- K portfolio containing p^* and feasible alternatives from distinct mechanism families. A plateau is detected when the best goal distance fails to improve beyond a preset threshold for several consecutive rounds. GoalEvolve then samples the next parent c from this portfolio according to

$$P_j(c) \propto \exp\left(-\frac{D_F(c) - D_F(p^*)}{T_j}\right) (1 + \beta N_c)(1 + \gamma A_c), \quad (4)$$

where N_c measures mechanism diversity and A_c measures alignment with the current bottleneck:

$$N_c = 1 - \frac{h(\phi_c)}{H}, \quad A_c = M_{\tau_b, \tau_c}. \quad (5)$$

Here, $h(\phi_c)$ counts how often the mechanism family ϕ_c was selected in the previous H inheritance decisions. The entry $M_{\tau_b, \tau_c} \in [0, 1]$ of a fixed tag compatibility matrix gives the compatibility between the candidate tag τ_c and the bottleneck tag τ_b , while β and γ

weight the diversity and alignment factors. At inheritance iteration j after plateau detection, GoalEvolve normalizes $P_j(c)$ over the top- K portfolio and samples one checkpoint. The temperature T_j is gradually reduced according to a fixed schedule as j increases. The champion is retained throughout this process, allowing GoalEvolve to explore another inheritance path without losing the best program found so far.

3.4 Post-Placement Instantiation

We instantiate GoalEvolve for post-placement timing and power optimization, where gate resizing, buffering, and VT reassignment improve a placed netlist, but routing-extracted RC parasitics may erode these gains. This gap between immediate improvement and downstream retention makes post-placement optimization a representative full-flow case study. All candidates start from the same placed netlist, while timing constraints, technology data, downstream routing, and the final evaluator remain fixed. The editable source scope is limited to Gate Resizer and Restructure mechanisms controlling candidate generation, scoring, scheduling, and commit decisions. The goal contract specifies post-route timing and power targets, with runtime as a soft constraint. White-box telemetry and structural statistics, including buffer growth and VT composition, provide diagnostic evidence.

Each full-flow evaluation records checkpoints after power reclaim, timing recovery, and supplemental repair. These observation contracts expose local effects, optimization debt, and final retention, allowing the Teacher to prioritize the next stage. The initial curriculum applies power reclaim before timing recovery, while structural RMP-A or RMP-T mechanisms become eligible only after cell-level optimization reaches a measured plateau. This instantiation maps GoalEvolve to a concrete source evolution task without imposing this stage sequence on the general framework.

4 Experiments

This section evaluates whether GoalEvolve attains frozen goals, retains gains after routing, and improves evolution efficiency across eight ASAP7 designs. Final QoR and checkpoint evidence are complemented by comparisons with equal-runtime repeated baselines and Codex goal mode. Evolution trajectories and an AES case study further link final improvements to the evolved mechanisms.

4.1 Experimental Setup

All experiments are conducted on a Rocky Linux 8.10 server with two Intel Xeon Platinum 8462Y+ processors and 314 GiB of memory. We evaluate GoalEvolve on eight post-placement designs spanning approximately 20 K–313 K instances. All designs use the ASAP7 7 nm predictive process design kit and its 7.5-track standard-cell library [5, 18]. Our implementation is based on OpenROAD [2] commit 08f67ee5.

Target construction is performed before evolution and is external to GoalEvolve. For each design, **preliminary experiments with a commercial reference flow** are used to profile its post-placement repair potential, establish an attainable QoR region, and identify the primary improvement opportunities across timing, leakage power, and dynamic power. The resulting targets require improvement in the primary metrics, while secondary QoR metrics

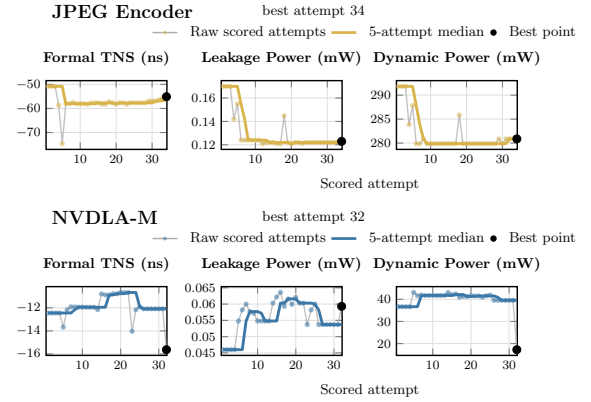


Figure 3: Timing and power trajectories across evolution attempts for JPEG (top) and NVDLA-M (bottom).

are constrained at the OpenROAD baseline or within a small relaxation band, and runtime is assigned a design-specific soft bound. All metric priorities and bounds are **frozen** before the first GoalEvolve or Codex goal mode run and reported in Table 1.

To ensure controlled evaluation, all candidate branches start from the same parent source snapshot and design checkpoint, but are built in isolated workspaces and evaluated with an identical checkpointed post-placement and global-routing flow. A GoalEvolve round comprises one Teacher and four coding agents, all using gpt-5.5 with high reasoning effort. Evolution terminates when the predefined goal is reached or the full-flow evaluation budget is exhausted.

4.2 Goal Attainment and QoR Improvement

Table 1 compares the OpenROAD baseline, predefined goal, and final champion to distinguish valid goal attainment from unconstrained baseline improvement.

GoalEvolve reaches all eight predefined goals: every final champion has $D_F = 0$ and passes the required validity checks. Relative to the OpenROAD baseline, the evolved programs improve post-route TNS by 28.83% on average and reduce leakage and dynamic power by 14.31% and 12.23%, respectively. Their achieved TNS values lie above the corresponding lower goal bounds by an average normalized margin of 21.67%, while leakage and dynamic power fall below their upper bounds by 3.65% and 2.85%, respectively. Complementing this endpoint comparison, Fig. 3 shows the timing and power trajectories of JPEG and NVDLA-M, exposing the tradeoffs behind their final champions.

Checkpoint evidence explains how these improvements persist. On NVDLA-M, power-oriented restructuring reduces total power from 39.6 to 16.5 mW, a 58.3% local reduction, of which 96.5% remains at the 17.3 mW routed endpoint. This move increases |TNS| from 12.09 to 22.64 ns; timing recovery lowers it to 13.33 ns, repaying 88.2% of the induced debt. Routing ends at 15.63 ns, retaining 7.01 ns of the 9.31 ns checkpoint repayment, or 75.3%. For the three designs primarily targeting power reduction (AES, NVDLA-A, and NVDLA-M), Table 2 reports 75.8% weighted retention of local power savings after routing. Thus, GoalEvolve favors mechanisms whose

Table 1: Comparison of the OpenROAD baseline, predefined goals, and GoalEvolve results (Baseline/Goal/Achieved).

Design	TNS (ns)			P_{leak} (μ W)			P_{dyn} (μ W)			Runtime (s)			Compliance
AES	-12.44/	$\geq$ -15.00/	-11.27	96.3/	$\leq$ 50.0/	45.2	431,903.7/	$\leq$ 385,000.0/	372,954.8	32/	$\leq$ 320/	260	Yes
Ariane	-7,568.43/	$\geq$ -3000.00/	-2,494.66	17,900.0/	$\leq$ 18,900.0/	18,000.0	640,100.0/	$\leq$ 645,000.0/	643,000.0	687/	$\leq$ 1374/	866	Yes
JPEG	-48.77/	$\geq$ -59.00/	-55.10	174.0/	$\leq$ 130.0/	123.0	293,826.0/	$\leq$ 285,000.0/	280,877.0	187/	$\leq$ 374/	324	Yes
MemPool	-3,680.33/	$\geq$ -3500.00/	-3,400.00	3,070.0/	$\leq$ 3,070.0/	3,020.0	275,930.0/	$\leq$ 250,000.0/	243,980.0	1,415/	$\leq$ 5,000/	4,563	Yes
NVDLA-A	-203.23/	$\geq$ -142.00/	-133.98	321.0/	$\leq$ 250.0/	235.0	164,679.0/	$\leq$ 154,000.0/	151,765.0	166/	$\leq$ 1000/	931	Yes
NVDLA-C	-55.58/	$\geq$ -38.5/	-11.92	17,300.0/	$\leq$ 17,300.0/	17,300.0	583,700.0/	$\leq$ 583,700.0/	583,700.0	632/	$\leq$ 1264/	1,206	Yes
NVDLA-M	-13.99/	$\geq$ -16.79/	-15.63	61.5/	$\leq$ 60.0/	59.3	44,538.5/	$\leq$ 20,000.0/	17,240.7	46/	$\leq$ 460/	392	Yes
NVDLA-P	-198.18/	$\geq$ -138.00/	-81.96	306.0/	$\leq$ 306.0/	304.0	39,694.0/	$\leq$ 40,000.0/	39,896.0	125/	$\leq$ 250/	110	Yes

Table 2: Power retention across optimization stages.

Design	Initial (mW)	Post-place (mW)	Post-route (mW)	Ret. (%)
AES	409.0	357.0	373.0	69.2
NVDLA-A	159.0	148.0	152.0	63.6
NVDLA-M	39.6	16.5	17.3	96.5
Weighted retention				75.8

Table 3: GoalEvolve (GE) vs. repeated baseline (RB).

Design	RB passes	TNS (ns) GE/RB	P_{leak} (μ W) GE/RB	P_{dyn} (mW) GE/RB
AES	9	-11.27/-12.90	45.2/101.0	372.955/435.899
NVDLA-A	6	-133.98/-198.32	235.0/343.0	151.765/164.657
NVDLA-C	3	-11.92/-55.16	17,300/17,300	583.700/583.700
NVDLA-M	9	-15.63/-17.69	59.3/79.5	17.241/50.721

Table 4: GoalEvolve versus Codex goal mode.

Design	GoalEvolve	Codex Goal Mode	Reduction
NVDLA-A	9	13	30.77%
NVDLA-M	32	49	34.69%
Average reduction			32.73%

benefits persist beyond immediate checkpoint gains to the routed endpoint.

To test whether goal attainment results merely from additional optimization time, we repeat the default baseline until its nominal cumulative runtime reaches or exceeds that of GoalEvolve, giving it 1.06–1.57 $\times$ the GoalEvolve budget over three to nine passes. Each pass reloads the preceding DEF and Verilog, reconstructs the constraints and RC model, estimates placement parasitics, and invokes the unchanged baseline operators. As Table 3 shows, GoalEvolve achieves higher (less negative) TNS on all four designs while reducing leakage and dynamic power by averages of 28.0% and 22.1%. These results indicate that GoalEvolve uses its additional computation to identify and implement effective source-level changes, rather than blindly retrying unchanged repair operators.

4.3 Comparison with Codex Goal Mode

To isolate the benefit of feedback tailored to physical design, we compare GoalEvolve with Codex operating in goal mode on NVDLA-A and NVDLA-M. Both methods start from the same source snapshot, use gpt-5.5, and share the same evaluator and frozen QoR goals. Codex receives these goals in a natural language prompt

and iterates between source revision and evaluation. We measure search cost by the number of valid formal evaluations, counting an evaluation only when it completes and returns all required metrics. For GoalEvolve, this count aggregates valid candidates produced by its parallel Students. A candidate reaches the goal only when its TNS satisfies the frozen lower bound and its leakage and dynamic power satisfy their frozen upper bounds.

GoalEvolve reduces the valid full-flow evaluation count relative to Codex by 30.77% on NVDLA-A and 34.69% on NVDLA-M, yielding an average reduction of 32.73% across the two designs. This result demonstrates higher evaluation efficiency under the same model, source snapshot, and evaluator.

4.4 Case Study: AES on ASAP7

On AES (ASAP7), GoalEvolve evolves a coordinated OpenROAD power-timing strategy beyond Tcl schedule retuning. First, stage-aware forced power reclaim evaluates cell-level VT reassignment and buffer removal with a unified utility combining leakage reduction, input-capacitance relief, near-critical-endpoint impact, and progress toward the target cell composition. Separate rankings use the same utility, with VT swaps attempted first. After parasitic and timing updates, each trial is retained only within candidate-specific TNS and WNS guards; otherwise, journaled rollback restores the netlist. Of 13,336 structural candidates, it commits 4,350 VT swaps and 1,249 buffer removals. Second, the evolved timing-recovery policy interleaves power reclaim with setup repair, using restored slack to offset power added by sizing and buffering. Sequential reclaim admits 232 additional precomputed VT swaps, with timing updated after each accepted move. Relative to the first formal AES record, the combined algorithm reduces leakage from 0.0891 to 0.0452 mW and dynamic power from 425.91 to 372.95 mW, while improving TNS from -12.56 to -11.27 ns. Together, these mechanisms expose structural power savings and restore timing without giving back the gain.

5 Conclusion

GoalEvolve reframes physical design algorithm development as goal-directed source evolution under full-flow feedback. Remaining QoR gaps identify the stage and source mechanism to revise, while checkpoint evidence and mechanism memory favor effects that survive downstream interactions. Experiments show that GoalEvolve reaches prescribed multi-metric goals more efficiently than Codex goal mode and improves post-route QoR over default OpenROAD. These results establish goal-guided source evolution as a practical methodology for physical design.

References

- [1] Ahmed Agiza and Sherief Reda. 2020. OpenPhySyn: An Open-Source Physical Synthesis Optimization Toolkit. In *2020 Workshop on Open-Source EDA Technology*. <https://woset-workshop.github.io/PDFs/2020/a09.pdf>
- [2] Tutu Ajayi, Vidya A. Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B. Kahng, Minsoo Kim, Jeongsup Lee, Uday Mallappa, Marina Neseem, Geraldo Pradipta, Sherief Reda, Mehdi Saligane, Sachin S. Sapatnekar, Carl Sechen, Mohamed Shalan, William Swartz, Lutong Wang, Zhehong Wang, Mingyu Woo, and Bangqi Xu. 2019. Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project. In *Proceedings of the 56th Annual Design Automation Conference*. ACM, 76:1–76:4. doi:10.1145/3316781.3326334
- [3] Chung-Kuan Cheng, Chester Holtz, Andrew B. Kahng, Bill Lin, and Uday Mallappa. 2023. DAGSizer: A Directed Graph Convolutional Network Approach to Discrete Gate Sizing of VLSI Graphs. *ACM Transactions on Design Automation of Electronic Systems* 28, 4 (2023), 52:1–52:31. doi:10.1145/3577019
- [4] Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang, and Lutong Wang. 2019. RePlace: Advancing Solution Quality and Routability Validation in Global Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 9 (2019), 1717–1730. doi:10.1109/TCAD.2018.2859220
- [5] Lawrence T. Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandrasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm FinFET Predictive Process Design Kit. *Microelectronics Journal* 53 (2016), 105–115. doi:10.1016/j.mejo.2016.04.006
- [6] Yufan Du, Zizheng Guo, Yibo Lin, Runsheng Wang, and Ru Huang. 2024. Fusion of Global Placement and Gate Sizing with Differentiable Optimization. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. ACM, 19:1–19:9. doi:10.1145/3676536.3676670
- [7] Amur Ghose, Junyeong Jang, Andrew B. Kahng, and Jakang Lee. 2026. Automated QoR Improvement in OpenROAD with Coding Agents. arXiv:2601.06268 <https://arxiv.org/abs/2601.06268>
- [8] Zizheng Guo and Yibo Lin. 2022. Differentiable-Timing-Driven Global Placement. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. ACM, 1315–1320. doi:10.1145/3489517.3530486
- [9] Taizun Jafri and Vidya A. Chhabria. 2026. GR-Evolve: Design-Adaptive Global Routing via LLM-Driven Algorithm Evolution. arXiv:2604.22234 <https://arxiv.org/abs/2604.22234>
- [10] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. 2026. ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=IKEdGCoDNC>
- [11] Yibo Lin, Zixuan Jiang, Jiaqi Gu, Wuxi Li, Shounak Dhar, Haoxing Ren, Bruce Khailany, and David Z. Pan. 2021. DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 4 (2021), 748–761. doi:10.1109/TCAD.2020.3003843
- [12] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 32201–32223. <https://proceedings.mlr.press/v235/liu24bs.html>
- [13] Jinwei Liu, Chak-Wa Pui, Fangzhou Wang, and Evangeline F. Y. Young. 2020. CUGR: Detailed-Routability-Driven 3D Global Routing with Probabilistic Resource Model. In *Proceedings of the 57th ACM/IEEE Design Automation Conference*. IEEE, 1–6. doi:10.1109/DAC18072.2020.9218646
- [14] Yi-Chen Lu, Siddhartha Nath, Vishal Khandelwal, and Sung Kyu Lim. 2021. RL-Sizer: VLSI Gate Sizing for Timing Optimization Using Deep Reinforcement Learning. In *Proceedings of the 58th ACM/IEEE Design Automation Conference*. IEEE, 733–738. doi:10.1109/DAC18074.2021.9586138
- [15] Alexander Novikov, Ngán Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv:2506.13131 <https://arxiv.org/abs/2506.13131>
- [16] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625, 7995 (2024), 468–475. doi:10.1038/s41586-023-06924-6
- [17] Niki van Stein and Thomas Bäck. 2025. LLaMEA: A Large Language Model Evolutionary Algorithm for Automatically Generating Metaheuristics. *IEEE Transactions on Evolutionary Computation* 29, 2 (2025), 331–345. doi:10.1109/TEVC.2024.3497793
- [18] Vinay Vashishtha, Manoj Vangala, and Lawrence T. Clark. 2017. ASAP7 Predictive Design Kit Development and Cell Design Technology Co-Optimization: Invited Paper. In *Proceedings of the 2017 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 992–998. doi:10.1109/ICCAD.2017.8203889
- [19] Xufeng Yao, Jiayi Jiang, Yuxuan Zhao, Peiyu Liao, Yibo Lin, and Bei Yu. 2026. EvoPlace: Evolution of Optimization Algorithms for Global Placement via Large Language Models. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2026). doi:10.1109/TCAD.2026.3652945 Early Access.
- [20] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. 2024. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. In *Advances in Neural Information Processing Systems*, Vol. 37. 43571–43608. doi:10.52202/079017-1381
- [21] Cunxi Yu and Haoxing Ren. 2026. Autonomous Evolution of EDA Tools: Multi-Agent Self-Evolved ABC. In *Proceedings of the 63rd ACM/IEEE Design Automation Conference* (Long Beach, CA, USA). ACM, New York, NY, USA, 7 pages. doi:10.1145/3770743.3804221 To appear.